

Лабораторная работа № 2

Создание структуры веб-приложения с помощью App Router. Проектирование авторизации

1. Место работы в курсе

Во второй лабораторной работе студент продолжает проект, созданный в лабораторной работе № 1, и превращает начальный каркас из двух страниц в связанное многостраничное веб-приложение.

Тематика приложения, проблема, целевая аудитория и основная предметная сущность сохраняются. Студент не копирует предметную область TheraCity, но использует сходный принцип построения: несколько разделов приложения работают с общей предметной идеей и образуют законченные пользовательские сценарии.

В TheraCity каталог app содержит разделы start, gallery, routes, cabinet, auth и about. В учебном примере StudySpace создаются разделы start, catalog, cabinet, auth и about. Студент должен заменить их эквивалентами собственной предметной области.

В этой работе создаются страницы входа и регистрации и учебное состояние текущего пользователя. Это прототип интерфейса, а не защищённая система: пароль не сверяется с базой данных, серверная сессия не создаётся, а состояние исчезает после перезагрузки страницы. PostgreSQL, Prisma ORM, хранение пользователей, хеширование паролей и серверная проверка прав будут реализованы в последующих лабораторных работах.

2. Цель работы

Разработать структуру многостраничного веб-приложения с помощью App Router, настроить общую навигацию, создать динамический маршрут предметной сущности и реализовать клиентский прототип интерфейса входа и регистрации с явным пониманием различий между аутентификацией, управлением сессией и авторизацией.

3. Задачи работы

В ходе выполнения работы студент должен:

- спроектировать набор разделов собственного приложения;
- создать страницы и вложенные маршруты в каталоге app;
- использовать общий корневой layout;
- настроить переходы через компонент Link;
- создать динамический маршрут для страницы отдельной сущности;
- разделить Server Components и Client Components;
- создать контекст учебного состояния пользователя;
- реализовать формы входа и регистрации;
- отобразить разные элементы интерфейса для гостя и учебного пользователя;
- объяснить ограничения клиентского прототипа авторизации;
- проверить проект с помощью ESLint и производственной сборки.

4. Теоретическая часть

4.1 App Router как файловый маршрутизатор

Официальная документация Next.js определяет App Router как файловый маршрутизатор. Каталоги задают сегменты URL, а специальные файлы задают поведение и содержимое маршрута. Страница создаётся файлом `page.tsx`, а общая разметка — файлом `layout.tsx`. (Next.js: App Router; Next.js: Layouts and Pages)

Пример соответствия файлов и адресов:

<code>app/page.tsx</code>	→ <code>/</code>
<code>app/start/page.tsx</code>	→ <code>/start</code>
<code>app/catalog/page.tsx</code>	→ <code>/catalog</code>
<code>app/auth/login/page.tsx</code>	→ <code>/auth/login</code>
<code>app/cabinet/page.tsx</code>	→ <code>/cabinet</code>

Каждый `page.tsx` должен экспортировать React-компонент по умолчанию:

```
export default function CatalogPage() {  
  return <h1>Каталог</h1>;  
}
```

Корневой `layout` обязателен. Он принимает `children` и оборачивает содержимое всех страниц. Общая навигация, провайдер состояния и повторяющиеся элементы оболочки приложения размещаются в `layout` или в компонентах, которые он подключает.

```
export default function RootLayout({  
  children,  
}: Readonly<{ children: React.ReactNode }>) {  
  return (  
    <html lang="ru">  
      <body>{children}</body>  
    </html>  
  );  
}
```

Согласно официальной документации Layouts and Pages, `layout` сохраняет состояние, остаётся интерактивным и не перерисовывается полностью при переходах между вложенными страницами. Это делает корневой `layout` подходящим местом для общей оболочки приложения.

4.2 Навигация между страницами

Компонент `Link` из `next/link` является основным средством перехода между маршрутами Next.js. Официальное руководство Linking and Navigating указывает, что `Link` поддерживает клиентские переходы и предварительную загрузку маршрутов. При переходе общая разметка сохраняется, а содержимое текущей страницы заменяется без полной перезагрузки документа.

```
import Link from "next/link";  
  
<Link href="/catalog">Открыть каталог</Link>
```

Обычный элемент `a` нужен для внешних ресурсов. Для внутренних страниц приложения в этой работе применяется `Link`.

Текущий путь в Client Component можно получить с помощью `usePathname` из `next/navigation`. В официальной справке Next.js `usePathname` описан как клиентский хук, возвращающий `pathname` текущего URL.

```
"use client";

import { usePathname } from "next/navigation";

export default function CurrentSection() {
  const pathname = usePathname();
  return <p>Текущий путь: {pathname}</p>;
}
```

В учебном приложении `usePathname` используется для `aria-current` и визуального выделения активного пункта навигации. Он не используется как средство защиты страницы.

4.3 Динамические сегменты

Если заранее неизвестны все адреса страниц предметных сущностей, Next.js позволяет создать динамический сегмент. Согласно официальной документации `Dynamic Route Segments`, имя каталога заключается в квадратные скобки.

```
app/catalog/[id]/page.tsx
```

Такой файл может обслуживать адреса `/catalog/library`, `/catalog/coworking` и другие значения `id`.

В актуальном App Router значение `params` является Promise. Его необходимо получить с помощью `await` в асинхронном компоненте страницы.

```
export default async function ItemPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  return <h1>Объект: {id}</h1>;
}
```

Значение динамического параметра приходит из URL и должно рассматриваться как внешнее значение. Если объект не найден, страница может вызвать `notFound` из `next/navigation`.

4.4 Server Components и Client Components

Официальная документация Next.js Server and Client Components указывает, что страницы и `layouts` в App Router по умолчанию являются Server Components. Они подходят для статической разметки, получения данных на сервере и сохранения секретов вне клиентского JavaScript.

Client Components применяются, когда необходимы состояние, обработчики событий, эффекты, пользовательские хуки или браузерные API. Граница клиентской части объявляется директивой "use client" перед импортами.

```
"use client";

import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

Директива не должна автоматически добавляться во все файлы. Официальная документация рекомендует оставлять статические части серверными и выделять интерактивность в небольшие Client Components. В этой работе страницы каталога остаются серверными, а навигация, провайдер пользователя и формы становятся клиентскими.

4.5 Аутентификация, сессия и авторизация

Официальное руководство Next.js Authentication разделяет защиту приложения на три понятия.

- Аутентификация подтверждает, что пользователь является тем, за кого себя выдаёт. Обычно пользователь предъявляет учётные данные, например email и пароль.
- Управление сессией сохраняет состояние входа между запросами.
- Авторизация определяет, к каким маршрутам, данным и действиям имеет доступ аутентифицированный пользователь.

Эти понятия связаны, но не взаимозаменяемы. Отображение формы входа ещё не выполняет аутентификацию. Запись имени в React state ещё не создаёт серверную сессию. Скрытие кнопки на клиенте ещё не запрещает обращение к защищённым данным.

В официальном руководстве Next.js показано, что обработка реальных учётных данных и серверная проверка выполняются в серверном окружении. Для сессионной cookie рекомендуются параметры HttpOnly, Secure, SameSite, Max-Age или Expires и Path. Защищённые операции должны повторно проверять права как можно ближе к источнику данных.

В лабораторной работе № 2 эти механизмы только изучаются. Реализуемый DemoAuthProvider хранит объект пользователя в памяти вкладки браузера, чтобы отработать условный интерфейс и передачу состояния. Он не принимает решение о реальном доступе к данным.

4.6 Контекст и состояние React

Официальная справка React определяет useState как хук, добавляющий переменную состояния в компонент. Вызов функции обновления изменяет состояние и запускает повторный рендер.

```
const [user, setUser] = useState<User | null>(null);
```

Контекст позволяет передавать значение глубоко по дереву компонентов без последовательной передачи props через каждый промежуточный компонент. Согласно официальной справке useContext, компонент получает значение ближайшего соответствующего провайдера выше по дереву и подписывается на его изменения.

```
const AuthContext = createContext<AuthContextValue | null>(null);
```

В корневом layout провайдер оборачивает страницы. Навигация, форма входа и личный кабинет получают один и тот же объект пользователя через useContext.

Контекст не является хранилищем базы данных и не является механизмом безопасности. После обновления страницы обычное состояние React создаётся заново. В этой работе такое поведение является намеренным признаком учебного прототипа.

4.7 Формы и проверка данных

Официальное руководство Next.js Forms указывает, что базовую клиентскую проверку можно выполнять HTML-атрибутами required и type="email". Серверная проверка нужна независимо от клиентской и будет добавлена позже.

```
<input name="email" type="email" required />
<input name="password" type="password" minLength={8} required />
```

Элемент label должен быть связан с полем. Ошибка должна выводиться текстом, а не только цветом. Для отправки используется обработчик onSubmit, который вызывает preventDefault в учебном клиентском прототипе.

```
function handleSubmit(event: React.FormEvent<HTMLFormElement>) {
  event.preventDefault();
  // Учебное обновление состояния.
}
```

Наличие minLength или собственной проверки на клиенте улучшает обратную связь, но не защищает сервер: HTTP-запрос можно сформировать без интерфейса. Поэтому в отчёте студент обязан явно отделить проверку удобства интерфейса от серверной проверки безопасности.

4.8 Связь с архитектурой TheraCity

В TheraCity корневой app/layout.tsx подключает AuthProvider и общий компонент BurgerMenu. Страницы app/auth/login/page.tsx и app/auth/register/page.tsx являются Client Components, используют состояние формы и получают функции из контекста. Реальный AuthProvider вызывает серверные маршруты /api/auth/login, /api/auth/me и /api/auth/logout.

В лабораторной работе № 2 используется только архитектурная аналогия: общий провайдер, формы и условный интерфейс. Вызовы API, cookie, Prisma и таблица пользователей не копируются, потому что соответствующая серверная инфраструктура ещё не создана.

5. Практическое задание

Этап 1. Проектирование разделов

Составьте карту будущих разделов собственного приложения. Должны быть предусмотрены:

- стартовая страница;
- каталог основных предметных сущностей;
- страница отдельной сущности;
- информационная страница;
- страница входа;
- страница регистрации;
- личный кабинет или другой раздел, предназначенный для пользователя.

Названия `catalog` и `cabinet` в примере не являются обязательными. Например, приложение о мероприятиях может использовать `events` и `profile`, приложение о рецептах — `recipes` и `account`, приложение о музейных объектах — `exhibits` и `cabinet`.

Подготовьте таблицу соответствия «назначение раздела — каталог app — URL». Эта таблица включается в отчёт.

Этап 2. Создание структуры App Router

Создайте каталоги и файлы по структуре примера из приложения к работе. В Git Bash из корня проекта можно выполнить:

```
mkdir -p app/start app/catalog/'[id]' app/cabinet app/auth/login
app/auth/register app/providers app/ui app/data
touch app/start/page.tsx app/catalog/page.tsx app/catalog/'[id]'/page.tsx
touch app/cabinet/page.tsx app/auth/login/page.tsx app/auth/register/page.tsx
touch app/providers/DemoAuthProvider.tsx app/ui/AppNav.tsx app/data/places.ts
```

В Windows каталоги можно создать через Explorer в Visual Studio Code. Квадратные скобки являются частью имени каталога `[id]`.

Сохраните существующие `app/about/page.tsx`, `app/globals.css`, `app/layout.tsx` и `app/page.tsx` из первой работы: их необходимо изменить, а не создавать второй проект.

Этап 3. Создание учебных данных

Создайте `app/data/places.ts`. Для примера StudySpace сущностью является место для учёбы. Массив содержит три объекта с уникальными строковыми `id`.

Для собственной темы замените тип `StudyPlace` и массив `studyPlaces`. У каждого объекта должны быть как минимум:

- уникальный идентификатор;
- название;
- краткое описание;

- одно дополнительное свойство, характерное для предметной области.

На этом этапе данные находятся в TypeScript-файле. Это не база данных. В лабораторных работах № 5 и № 6 массив будет заменён моделями Prisma и записями PostgreSQL.

Этап 4. Каталог и динамическая страница

Создайте `app/catalog/page.tsx`. Страница выводит карточки из массива и формирует `Link` на `/catalog/{id}`.

Создайте `app/catalog/[id]/page.tsx`. Получите `id` через `await params`, найдите объект и вызовите `notFound`, если объект отсутствует.

Проверьте минимум четыре адреса:

```
http://localhost:3000/catalog
http://localhost:3000/catalog/library
http://localhost:3000/catalog/coworking
http://localhost:3000/catalog/unknown
```

Для `unknown` должна отображаться стандартная страница «Not Found», а не пустая разметка и не ошибка выполнения.

Этап 5. Общая навигация

Создайте `app/ui/AppNav.tsx` как Client Component. Добавьте `Link` на основные разделы, `usePathname` для активного пункта и `aria-current="page"` для текущего маршрута.

Подключите `AppNav` в `app/layout.tsx`. Навигация должна отображаться на всех страницах. Не копируйте один и тот же `header` вручную в каждый `page.tsx`.

Измените `app/page.tsx` так, чтобы адрес / перенаправлял на `/start` с помощью `redirect` из `next/navigation`. Этот приём используется и в `TheraCity`.

Этап 6. DemoAuthProvider

Создайте `app/providers/DemoAuthProvider.tsx`. Провайдер должен хранить `User | null` и предоставлять функции `login`, `register` и `logout`.

Требования к прототипу:

- объект пользователя содержит `name` и `email`;
- `login` создаёт учебного пользователя из `email`;
- `register` создаёт пользователя из имени и `email`;
- `logout` устанавливает `user` в `null`;
- хук `useDemoAuth` сообщает об ошибке, если используется вне провайдера;
- пароль не сохраняется ни в `state` провайдера, ни в `localStorage`, ни в файле.

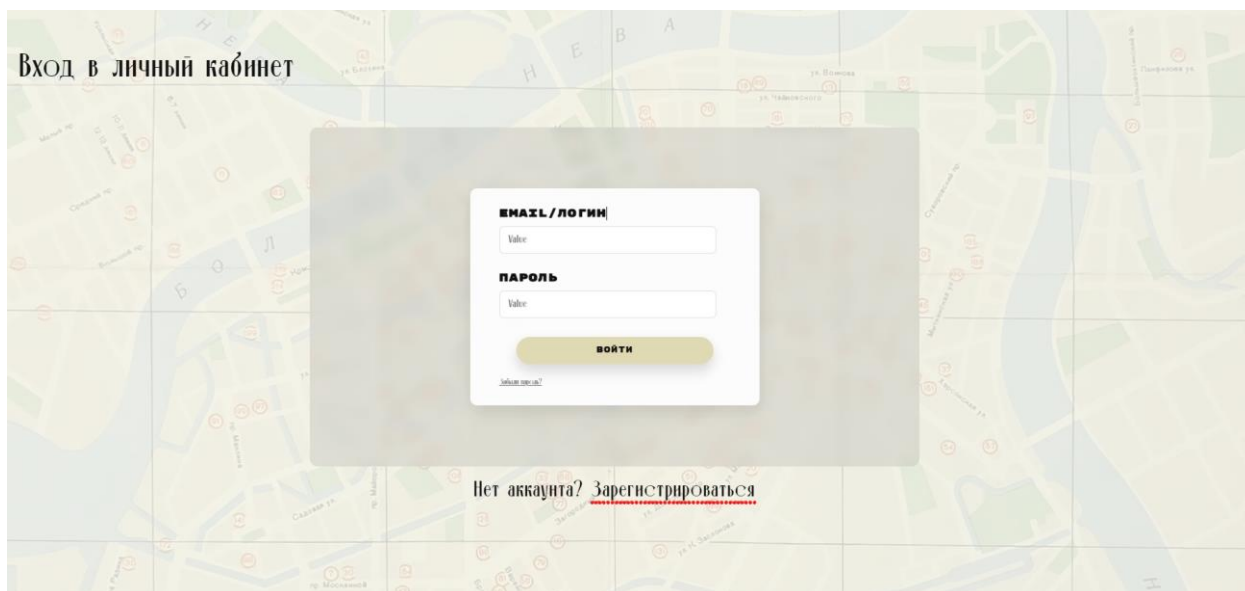
Оберните содержимое `body` провайдером в корневом `layout`.

Этап 7. Форма входа

Создайте `app/auth/login/page.tsx`. Страница должна:

- быть Client Component;
- содержать связанные `label` и `input`;
- использовать `type="email"` и `type="password"`;
- хранить введённые значения в `useState`;
- проверять длину пароля для обратной связи;
- вызывать демо-функцию `login`;
- после успешной учебной проверки переходить в `/cabinet`;
- содержать ссылку на `/auth/register`;
- показывать предупреждение, что вход является прототипом.

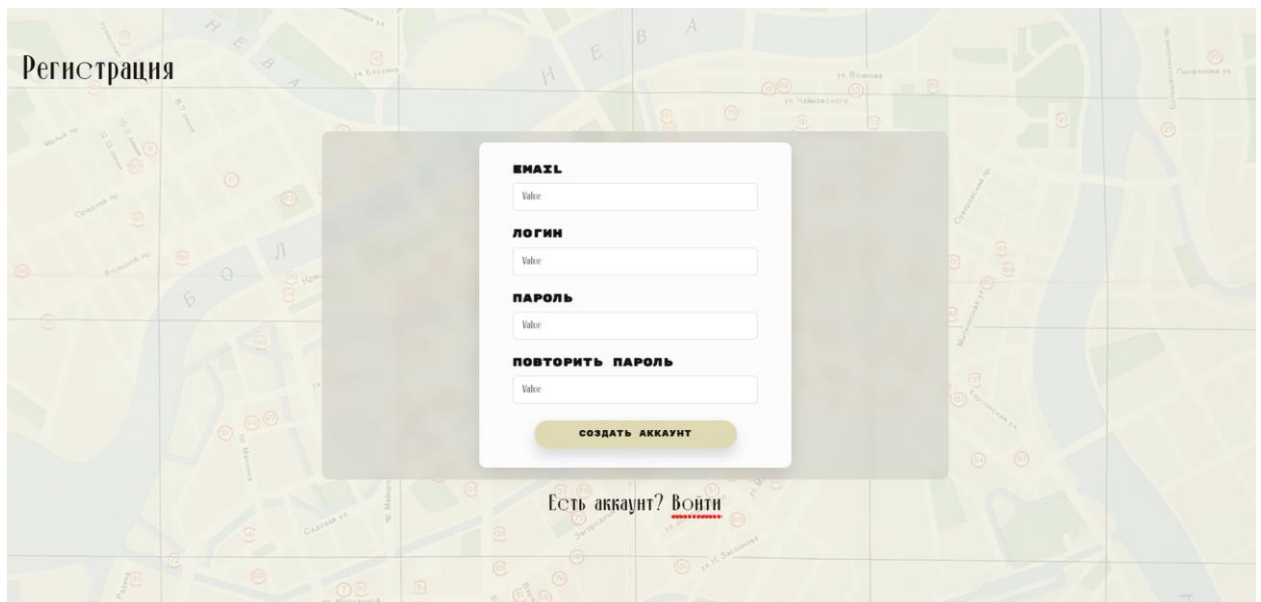
Нельзя писать в интерфейсе, что пароль «проверен сервером» или что пользователь «безопасно авторизован».



Этап 8. Форма регистрации

Создайте `app/auth/register/page.tsx`. Добавьте поля имени, email, пароля и повтора пароля. Проверьте совпадение паролей и минимальную длину.

После учебной проверки вызовите `register` и выполните переход в `/cabinet`. Запись пользователя в базу данных в этой работе не выполняется.



Этап 9. Личный кабинет и условный интерфейс

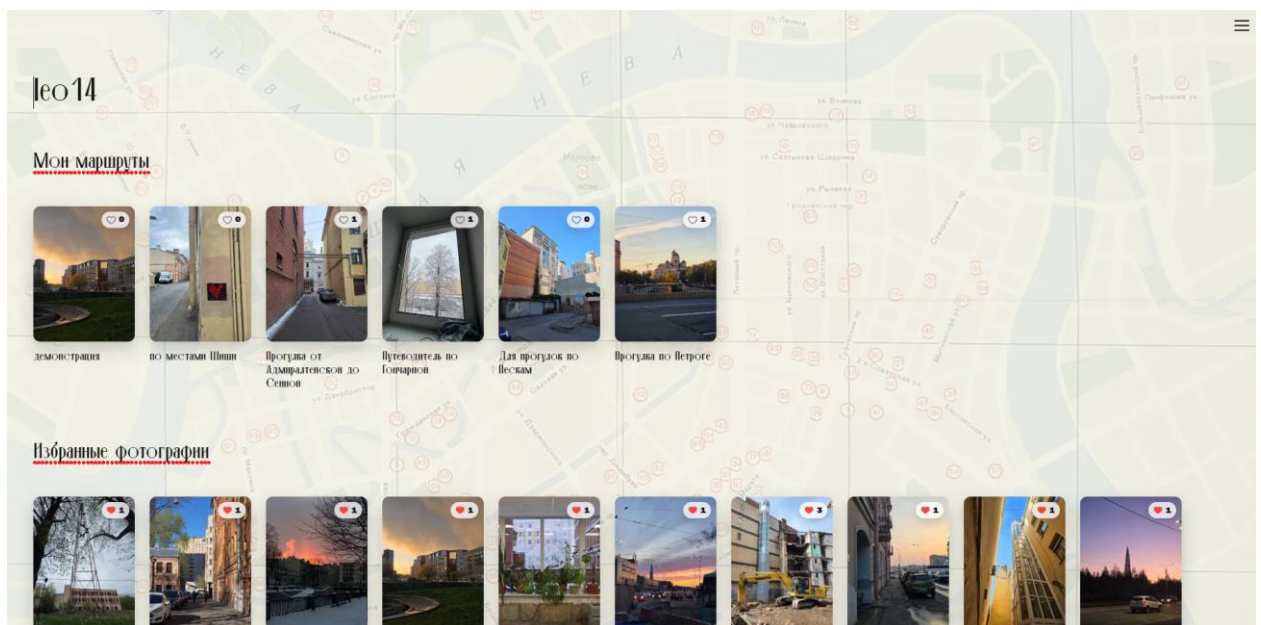
Создайте `app/cabinet/page.tsx` как Client Component.

Если `user` равен `null`, выведите сообщение и `Link` на страницу входа. Если пользователь существует, выведите его имя, `email` и кнопку «Выйти».

В `AppNav` измените последний элемент:

- для гостя показывается `Link` «Войти»;
- для учебного пользователя показывается его имя и кнопка «Выйти».

Это условный интерфейс. Страница `/cabinet` остаётся доступной по прямому URL и сама выбирает, что показать. Серверная защита появится после реализации сессии.



Этап 10. Стилизация

Обновите app/globals.css. Создайте визуально различимые состояния:

- активный пункт навигации;
- карточка каталога;
- поле формы;
- основная кнопка;
- сообщение об ошибке;
- предупреждение о демонстрационном режиме.

Полная адаптивность не является задачей этой работы. Она будет реализована и проверена в лабораторной работе № 4. Интерфейс всё же не должен терять содержимое при обычной ширине окна браузера.

Этап 11. Проверка пользовательских сценариев

Запустите проект:

```
npm run dev
```

Проверьте сценарий гостя:

- открыть /start;
- перейти в каталог;
- открыть две динамические страницы;
- перейти в кабинет;
- открыть форму входа;
- увидеть ошибку для короткого пароля;
- выполнить учебный вход;
- увидеть пользователя в навигации и кабинете;
- нажать «Выйти»;
- обновить страницу и убедиться, что состояние прототипа сбрасывается.

Проверьте сценарий регистрации аналогичным образом.

Остановите сервер сочетанием Ctrl + C, затем выполните:

```
npm run lint  
npm run build
```

Если установленная версия create-next-app не содержит сценарий lint, используйте команду `pnpm run lint` или добавьте сценарий "lint": "eslint" в package.json.

6. Требования к результату

После выполнения работы студент должен предоставить проект, в котором:

- продолжается тема лабораторной работы № 1;
- создано не менее пяти содержательных маршрутов;

- используется общий корневой layout;
- внутренняя навигация выполнена через next/link;
- активный раздел отмечен с помощью usePathname и aria-current;
- существует динамический маршрут [id] или [slug];
- неизвестный идентификатор обрабатывается через notFound;
- статические страницы остаются Server Components;
- интерактивные компоненты имеют обоснованную директиву "use client";
- создан DemoAuthProvider с типизированным контекстом;
- реализованы формы входа и регистрации;
- личный кабинет отображает состояния гостя и учебного пользователя;
- пароль не сохраняется в клиентское хранилище;
- в интерфейсе указано, что авторизация является прототипом;
- npm run lint и npm run build завершаются без ошибок.

Запрещается выдавать клиентский прототип за защищённую авторизацию. Нельзя использовать условие `user !== null` как единственную защиту будущих данных или операций.

7. Содержание отчёта

Отчёт должен содержать:

- название и цель работы;
- краткое описание продолжения выбранной темы;
- таблицу «назначение раздела — каталог app — URL»;
- скриншот структуры каталога app;
- объяснение файлов page.tsx и layout.tsx;
- объяснение динамического сегмента;
- код типа предметной сущности и массива учебных данных;
- код страницы каталога;
- код динамической страницы;
- код DemoAuthProvider;
- код форм входа и регистрации;
- скриншот каталога;
- скриншот динамической страницы;
- скриншот формы входа с ошибкой;
- скриншот личного кабинета после учебного входа;
- результаты npm run lint и npm run build;
- таблицу различий между аутентификацией, сессией и авторизацией;
- объяснение, почему реализованный прототип не является средством безопасности;
- ответы на контрольные вопросы;
- вывод по работе.

Контрольные вопросы:

- Как каталог app преобразуется в URL?
- Чем page.tsx отличается от layout.tsx?
- Для чего используются квадратные скобки в имени каталога?
- Почему params в актуальном App Router получают через await?
- Когда требуется директива "use client"?
- Какие преимущества даёт Link по сравнению с полной перезагрузкой?
- Чем аутентификация отличается от авторизации?
- Для чего нужна сессия?
- Почему скрытая кнопка не защищает серверную операцию?
- Почему DemoAuthProvider сбрасывается после обновления страницы?
- Какие части прототипа будут заменены после подключения PostgreSQL и Prisma?

8. Источники

Теоретическая часть подготовлена по официальной документации. Практические примеры адаптированы к последовательности курса и архитектурным приёмам проекта TheraCity.

- Next.js: App Router — <https://nextjs.org/docs/app>
- Next.js: Layouts and Pages — <https://nextjs.org/docs/app/getting-started/layouts-and-pages>
- Next.js: Linking and Navigating — <https://nextjs.org/docs/app/getting-started/linking-and-navigating>
- Next.js: Dynamic Route Segments — <https://nextjs.org/docs/app/api-reference/file-conventions/dynamic-routes>
- Next.js: Server and Client Components — <https://nextjs.org/docs/app/getting-started/server-and-client-components>
- Next.js: usePathname — <https://nextjs.org/docs/app/api-reference/functions/use-pathname>
- Next.js: Authentication — <https://nextjs.org/docs/app/guides/authentication>
- Next.js: Forms — <https://nextjs.org/docs/app/guides/forms>
- React: useState — <https://react.dev/reference/react/useState>
- React: useContext — <https://react.dev/reference/react/useContext>
- React: createContext — <https://react.dev/reference/react/createContext>

КОД ДЛЯ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ № 2

1. Итоговая структура файлов (этап 2)

Показывает структуру маршрутов, компонентов, данных и провайдера состояния.

```

app/
├── about/
│   └── page.tsx
├── auth/
│   ├── login/
│   │   └── page.tsx
│   └── register/
│       └── page.tsx
├── cabinet/
│   └── page.tsx
├── catalog/
│   ├── [id]/
│   │   └── page.tsx
│   └── page.tsx
├── data/
│   └── places.ts
├── providers/
│   └── DemoAuthProvider.tsx
├── start/
│   └── page.tsx
├── ui/
│   └── AppNav.tsx
├── globals.css
├── layout.tsx
└── page.tsx

```

2. Файл app/data/places.ts (этап 3)

```

// Описывает учебные места, демонстрационные данные и поиск объекта по id.

// Описываем поля одной предметной сущности.
export type StudyPlace = {
  id: string;
  title: string;
  summary: string;
  address: string;
};

// Храним демонстрационный набор объектов в типизированном массиве.
export const studyPlaces: StudyPlace[] = [
  {
    id: "library",
    title: "Научная библиотека",
    summary: "Тихое место для индивидуальной работы с книгами и конспектами.",
    address: "Университетская улица, 1",
  },
  {
    id: "coworking",
    title: "Студенческий коворкинг",
    summary: "Пространство для групповых проектов и обсуждений.",
    address: "Учебный корпус, аудитория 210",
  },
  {
    id: "campus",
    title: "Открытая площадка кампуса",
    summary: "Место для чтения и коротких учебных встреч на свежем воздухе.",
    address: "Центральный двор кампуса",
  },
];

```

```

    },
  ];

  export function getStudyPlace(id: string): StudyPlace | undefined {
    return studyPlaces.find((place) => place.id === id);
  }

```

3. Файл app/providers/DemoAuthProvider.tsx (этап 6)

```

// Хранит учебное состояние пользователя и предоставляет вход и выход через React
Context.

"use client";

import {
  createContext,
  useContext,
  useMemo,
  useState,
  type ReactNode,
} from "react";

// Описываем данные учебного пользователя.
export type DemoUser = {
  name: string;
  email: string;
};

// Задаём контракт данных и функций контекста.
type DemoAuthContextValue = {
  user: DemoUser | null;
  login: (email: string) => void;
  register: (name: string, email: string) => void;
  logout: () => void;
};

const DemoAuthContext = createContext<DemoAuthContextValue | null>(null);

// Объявляем основной компонент этого файла.
export default function DemoAuthProvider({
  children,
}: Readonly<{ children: ReactNode }>) {
  // Создаём локальное состояние компонента.
  const [user, setUser] = useState<DemoUser | null>(null);

  // Мемоизируем объект контекста до изменения его зависимостей.
  const value = useMemo<DemoAuthContextValue>(
    () => ({
      user,
      login(email) {
        const name = email.split("@")[0]?.trim() || "Студент";
        setUser({ name, email });
      },
      register(name, email) {
        setUser({ name: name.trim(), email });
      },
      logout() {
        setUser(null);
      },
    }),
    [user],
  );

  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <DemoAuthContext.Provider value={value}>
      {children}
    </DemoAuthContext.Provider>
  );
}

```

```

    </DemoAuthContext.Provider>
  );
}

// Возвращаем контекст и защищаем хук от вызова вне провайдера.
export function useDemoAuth(): DemoAuthContextValue {
  const context = useContext(DemoAuthContext);

  if (!context) {
    throw new Error("useDemoAuth должен использоваться внутри DemoAuthProvider");
  }

  return context;
}

```

4. Файл app/layout.tsx (этапы 5–6)

```

// Подключает общие стили, навигацию и DemoAuthProvider ко всем страницам.

import type { Metadata } from "next";
import type { ReactNode } from "react";
import DemoAuthProvider from "../providers/DemoAuthProvider";
import AppNav from "../ui/AppNav";
import "../globals.css";

// Задаём заголовок и описание страницы для Metadata API.
export const metadata: Metadata = {
  title: "StudySpace",
  description: "Учебный проект о местах для самостоятельной работы",
};

// Объявляем основной компонент этого файла.
export default function RootLayout({
  children,
}: Readonly<{ children: ReactNode }>) {
  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <html lang="ru">
      <body>
        <DemoAuthProvider>
          <AppNav />
          {children}
        </DemoAuthProvider>
      </body>
    </html>
  );
}

```

5. Файл app/ui/AppNav.tsx (этапы 5 и 9)

```

// Формирует навигацию, отмечает активный маршрут и показывает состояние пользователя.

"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import { useDemoAuth } from "../../providers/DemoAuthProvider";

// Описываем пункты меню единым массивом.
const links = [
  { href: "/start", label: "Главная" },
  { href: "/catalog", label: "Каталог" },
  { href: "/about", label: "О проекте" },
  { href: "/cabinet", label: "Кабинет" },
];

```

```

function isActive(pathname: string, href: string): boolean {
  return pathname === href || pathname.startsWith(`${href}/`);
}

// Объявляем основной компонент этого файла.
export default function AppNav() {
  const pathname = usePathname();
  const { user, logout } = useDemoAuth();

  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <header className="site-header">
      <Link className="brand" href="/start">
        StudySpace
      </Link>

      <nav aria-label="Основная навигация">
        <ul className="nav-list">
          {links.map((link) => (
            <li key={link.href}>
              <Link
                className="nav-link"
                href={link.href}
                aria-current={isActive(pathname, link.href) ? "page" : undefined}
              >
                {link.label}
              </Link>
            </li>
          ))}
        </ul>
      </nav>

      {user ? (
        <div className="auth-summary">
          <span>{user.name}</span>
          <button type="button" onClick={logout}>
            Выйти
          </button>
        </div>
      ) : (
        <Link className="login-link" href="/auth/login">
          Войти
        </Link>
      )}
    </header>
  );
}

```

6. Файл app/page.tsx (этап 5)

```

// Перенаправляет корневой маршрут приложения на стартовую страницу.
import { redirect } from "next/navigation";

// Объявляем основной компонент этого файла.
export default function HomePage() {
  // Выполняем серверное перенаправление на нужный маршрут.
  redirect("/start");
}

```

7. Файл app/start/page.tsx (этап 5)

```

// Выводит стартовую страницу и ссылки на основные пользовательские сценарии.
import Link from "next/link";

```



```
// Объявляем основной компонент этого файла.
export default function StartPage() {
  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <main>
      <p className="eyebrow">Учебный проект</p>
      <h1>StudySpace</h1>
      <p className="lead">
        Каталог мест для самостоятельной и совместной работы студентов.
      </p>
      <Link className="primary-link" href="/catalog">
        Открыть каталог
      </Link>
    </main>
  );
}
```

8. Файл app/catalog/page.tsx (этап 4)

```
// Строит каталог карточек из массива учебных мест.

import Link from "next/link";
import { studyPlaces } from "../data/places";

// Объявляем основной компонент этого файла.
export default function CatalogPage() {
  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <main>
      <h1>Каталог учебных мест</h1>
      <div className="card-grid">
        {studyPlaces.map((place) => (
          <article className="card" key={place.id}>
            <h2>{place.title}</h2>
            <p>{place.summary}</p>
            <Link href={`/${catalog}/${place.id}`}>Подробнее</Link>
          </article>
        ))}
      </div>
    </main>
  );
}
```

9. Файл app/catalog/[id]/page.tsx (этап 4)

```
// Получает динамический id, находит объект и выводит страницу либо 404.

import Link from "next/link";
import { notFound } from "next/navigation";
import { getStudyPlace } from "../../data/places";

// Типизируем параметры динамического маршрута.
type PlacePageProps = {
  params: Promise<{ id: string }>;
};

// Объявляем основной компонент этого файла.
export default async function PlacePage({ params }: PlacePageProps) {
  // Получаем id из параметров динамического маршрута.
  const { id } = await params;
  // Ищем объект с идентификатором из URL.
  const place = getStudyPlace(id);

  if (!place) {
    // Показываем стандартную страницу 404, если объект не найден.
    notFound();
  }
}
```

```

    }

    // Возвращаем JSX-разметку, которую React выведет на странице.
    return (
      <main>
        <p className="eyebrow">Учебное место</p>
        <h1>{place.title}</h1>
        <p className="lead">{place.summary}</p>
        <p>
          <strong>Адрес:</strong> {place.address}
        </p>
        <Link href="/catalog">Вернуться в каталог</Link>
      </main>
    );
  }
}

```

10. Файл app/auth/login/page.tsx (этап 7)

```

// Обрабатывает учебный вход и переводит пользователя в личный кабинет.

"use client";

import Link from "next/link";
import { useRouter } from "next/navigation";
import { useState, type FormEvent } from "react";
import { useDemoAuth } from "../../providers/DemoAuthProvider";

// Объявляем основной компонент этого файла.
export default function LoginPage() {
  const router = useRouter();
  const { login } = useDemoAuth();
  // Создаём локальное состояние компонента.
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");

  // Обрабатываем отправку формы без перезагрузки страницы.
  function handleSubmit(event: FormEvent<HTMLFormElement>) {
    // Отменяем стандартное действие события и выполняем собственную логику.
    event.preventDefault();
    setError("");

    if (password.length < 8) {
      setError("Введите не менее 8 символов.");
      return;
    }

    login(email.trim());
    // После успешного действия переходим на следующий маршрут.
    router.push("/cabinet");
  }

  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <main className="form-page">
      <h1>Вход</h1>
      <p className="demo-warning">
        Учебный прототип: пароль не проверяется сервером и не сохраняется.
      </p>

      <form className="auth-form" onSubmit={handleSubmit}>
        <label htmlFor="email">Email</label>
        <input
          id="email"
          type="email"
          value={email}

```

```

        onChange={(event) => setEmail(event.target.value)}
        autoComplete="email"
        required
      />

      <label htmlFor="password">Пароль</label>
      <input
        id="password"
        type="password"
        value={password}
        onChange={(event) => setPassword(event.target.value)}
        autoComplete="current-password"
        minLength={8}
        required
      />

      {error ? <p className="form-error">{error}</p> : null}

      <button type="submit">Войти в демонстрационном режиме</button>
    </form>

    <p>
      Нет учебной записи? <Link href="/auth/register">Регистрация</Link>
    </p>
  </main>
);
}

```

11. Файл app/auth/register/page.tsx (этап 8)

```

// Создаёт учебного пользователя после отправки формы регистрации.

"use client";

import Link from "next/link";
import { useRouter } from "next/navigation";
import { useState, type FormEvent } from "react";
import { useDemoAuth } from "../../providers/DemoAuthProvider";

// Объявляем основной компонент этого файла.
export default function RegisterPage() {
  const router = useRouter();
  const { register } = useDemoAuth();
  // Создаём локальное состояние компонента.
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [confirmation, setConfirmation] = useState("");
  const [error, setError] = useState("");

  // Обрабатываем отправку формы без перезагрузки страницы.
  function handleSubmit(event: FormEvent<HTMLFormElement>) {
    // Отменяем стандартное действие события и выполняем собственную логику.
    event.preventDefault();
    setError("");

    if (password.length < 8) {
      setError("Пароль должен содержать не менее 8 символов.");
      return;
    }

    if (password !== confirmation) {
      setError("Введённые пароли не совпадают.");
      return;
    }

    register(name, email.trim());
  }
}

```

```

    // После успешного действия переходим на следующий маршрут.
    router.push("/cabinet");
  }

  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <main className="form-page">
      <h1>Регистрация</h1>
      <p className="demo-warning">
        Данные существуют только в состоянии текущей вкладки и не записываются в БД.
      </p>

      <form className="auth-form" onSubmit={handleSubmit}>
        <label htmlFor="name">Имя</label>
        <input
          id="name"
          value={name}
          onChange={(event) => setName(event.target.value)}
          autoComplete="name"
          required
        />

        <label htmlFor="email">Email</label>
        <input
          id="email"
          type="email"
          value={email}
          onChange={(event) => setEmail(event.target.value)}
          autoComplete="email"
          required
        />

        <label htmlFor="password">Пароль</label>
        <input
          id="password"
          type="password"
          value={password}
          onChange={(event) => setPassword(event.target.value)}
          autoComplete="new-password"
          minLength={8}
          required
        />

        <label htmlFor="confirmation">Повторите пароль</label>
        <input
          id="confirmation"
          type="password"
          value={confirmation}
          onChange={(event) => setConfirmation(event.target.value)}
          autoComplete="new-password"
          minLength={8}
          required
        />

        {error ? <p className="form-error">{error}</p> : null}

        <button type="submit">Создать учебную запись</button>
      </form>

      <p>
        Уже есть учебная запись? <Link href="/auth/login">Войти</Link>
      </p>
    </main>
  );
}

```

12. Файл app/cabinet/page.tsx (этап 9)

```

// Показывает гостевой экран либо сведения о вошедшем пользователе.

"use client";

import Link from "next/link";
import { useDemoAuth } from "../providers/DemoAuthProvider";

// Объявляем основной компонент этого файла.
export default function CabinetPage() {
  const { user, logout } = useDemoAuth();

  // Разделяем интерфейс гостя и вошедшего пользователя.
  if (!user) {
    // Возвращаем JSX-разметку, которую React выведет на странице.
    return (
      <main>
        <h1>Личный кабинет</h1>
        <p>В демонстрационном состоянии пользователь отсутствует.</p>
        <Link href="/auth/login">Перейти ко входу</Link>
      </main>
    );
  }

  return (
    <main>
      <h1>Личный кабинет</h1>
      <dl className="profile-list">
        <dt>Имя</dt>
        <dd>{user.name}</dd>
        <dt>Email</dt>
        <dd>{user.email}</dd>
      </dl>
      <button type="button" onClick={logout}>
        Выйти
      </button>
    </main>
  );
}

```

13. Файл app/globals.css (этап 10)

```

/* Задаёт общее оформление навигации, форм, карточек и адаптивной сетки. */

/* Включаем предсказуемый расчёт размеров элементов. */
* {
  box-sizing: border-box;
}

html {
  background: #f4f2ed;
}

/* Задаём базовые параметры страницы и типографики. */
body {
  margin: 0;
  color: #1f2933;
  font-family: Arial, Helvetica, sans-serif;
}

a {
  color: #184e77;
}

button,
input {
  font: inherit;
}

```

```

button {
  cursor: pointer;
}

/* Ограничиваем ширину основного содержимого и выравниваем его по центру. */
main {
  width: min(960px, calc(100% - 32px));
  margin: 0 auto;
  padding: 48px 0;
}

.site-header {
  display: flex;
  align-items: center;
  gap: 24px;
  min-height: 72px;
  padding: 12px 24px;
  border-bottom: 1px solid #d7d2c8;
  background: #ffffff;
}

.brand {
  color: #1f2933;
  font-size: 22px;
  font-weight: 800;
  text-decoration: none;
}

.nav-list {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  margin: 0;
  padding: 0;
  list-style: none;
}

.nav-link,
.login-link {
  display: inline-block;
  padding: 8px 10px;
  border-radius: 8px;
  text-decoration: none;
}

.nav-link[aria-current="page"] {
  background: #dbeafe;
  color: #123b5d;
  font-weight: 700;
}

.auth-summary {
  display: flex;
  align-items: center;
  gap: 10px;
  margin-left: auto;
}

.eyebrow {
  margin-bottom: 8px;
  font-weight: 800;
  letter-spacing: 0.08em;
  text-transform: uppercase;
}

.lead {
  max-width: 680px;

```

```

    font-size: 20px;
    line-height: 1.6;
}

.primary-link {
    display: inline-block;
    margin-top: 16px;
    padding: 12px 18px;
    border-radius: 10px;
    background: #184e77;
    color: #ffffff;
    font-weight: 700;
    text-decoration: none;
}

.card-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
    gap: 18px;
}

/* Оформляем интерактивную карточку объекта. */
.card {
    padding: 22px;
    border: 1px solid #d7d2c8;
    border-radius: 14px;
    background: #ffffff;
}

.card h2 {
    margin-top: 0;
}

.form-page {
    max-width: 620px;
}

.auth-form {
    display: grid;
    gap: 10px;
    margin: 24px 0;
    padding: 24px;
    border-radius: 14px;
    background: #ffffff;
}

.auth-form input {
    min-height: 44px;
    padding: 8px 10px;
    border: 1px solid #8d99a6;
    border-radius: 8px;
}

.auth-form button {
    min-height: 44px;
    margin-top: 10px;
    border: 0;
    border-radius: 8px;
    background: #184e77;
    color: #ffffff;
    font-weight: 700;
}

.demo-warning {
    padding: 12px 14px;
    border-left: 4px solid #c67c00;
    background: #fff4d6;
}

```

```

.form-error {
  color: #9b1c1c;
  font-weight: 700;
}

.profile-list {
  display: grid;
  grid-template-columns: 100px 1fr;
  gap: 8px 16px;
  max-width: 520px;
}

.profile-list dt {
  font-weight: 700;
}

.profile-list dd {
  margin: 0;
}

```

14. Что студент должен заменить

- название StudySpace;
- тип StudyPlace и его свойства;
- три учебных объекта;
- названия и URL предметных разделов;
- тексты стартовой страницы;
- подписи навигации;
- поля, которые уместны в личном кабинете;
- цветовое оформление.

Нельзя удалять предупреждение о демонстрационном режиме до появления реальной серверной аутентификации. После замены предметной области должны сохраниться App Router, динамический сегмент, типизация, общий layout и разделение серверных и клиентских компонентов.